Evaluating Machine-Readable HCI Skills for LLM-Generated User Interfaces

An Agentic Experimental Framework for Testing Quality, Consistency, and Cross-Model Generalization

Nathan Conklin
nathan.conklin@vt.edu
Department of Computer Science,
Virginia Tech

Abstract

Large language models can now generate executable user interfaces from natural-language task descriptions. This capability shifts an important HCI question from whether an interface can be generated to whether the generation process can apply established design knowledge consistently. Prior work proposed machine-readable HCI Skills as procedural specifications that encode principles such as Nielsen's heuristics, Norman's affordance prescriptions, WCAG requirements, cognitive-load constraints, mixed-initiative interaction, direct manipulation, and ability-based personalization [1]. The central claim remains empirical: do these Skills measurably improve the interfaces that LLMs generate?

This whitepaper defines an agentic study that can answer that question without a human-subject experiment. The proposed design uses a controlled corpus of approximately ten UI tasks, three foundation-model families, three HCI-guidance conditions, and repeated generations. A reference design with five replications per cell produces 450 generated interfaces. Each interface is evaluated through the same blinded pipeline using deterministic checks where possible and structured independent agent judges for criteria that require semantic interpretation. The study measures HCI conformance, violation counts, severity, and generation-to-generation variance. Four research questions test the effectiveness of Skills, their advantage over conventional HCI prompting, their effect on consistency, and their generalizability across models and task types. The study turns the vision of procedural HCI into a falsifiable experimental program while keeping the claims bounded to measurable interface conformance rather than human usability outcomes.

1. From a Vision Claim to an Empirical Claim

Automating the Application of HCI Principles proposes a future in which HCI knowledge is encoded as machine-readable `skill.md` artifacts that a generating agent can load at runtime [1]. The paper argues that this mechanism can make accessibility, learnability, consistency, and related design properties part of the generation process. It also characterizes Skills as uniform, inspectable, editable, and verifiable. Those properties make the proposal suitable for controlled evaluation.

The vision paper does not yet establish whether Skill-conditioned generation produces better interfaces. Its primary contribution is a framework and research agenda. A scientific follow-on study should isolate a narrower claim, manipulate the presence and form of HCI guidance, measure the resulting interfaces, and test whether observed differences exceed the variation expected from stochastic generation.

The FutureHCI paper *Can LLMs Improve Accessibility?* provides a useful methodological precedent [2]. That study defined controlled accessibility defects, evaluated multiple LLMs under different prompting conditions, produced 360 scored outputs, and analyzed differences with a fixed rubric. The proposed HCI Skills study applies the same general experimental logic to interface generation. Instead of asking whether an LLM can detect and repair a known WCAG violation, it asks whether procedural HCI knowledge changes what the LLM generates in the first place.

The contribution of the proposed study is therefore specific: **a controlled, agentic evaluation framework for testing whether machine-readable HCI Skills improve the measured HCI quality and consistency of LLM-generated user interfaces across models and tasks.**

2. Research Questions and Hypotheses

The study should use one dataset to answer four related research questions. Each question tests a distinct part of the Skills argument.

2.1 RQ1: Effectiveness

RQ1: Does applying machine-readable HCI Skills during generation improve the HCI quality of LLM-generated user interfaces?

The primary comparison is between a baseline condition with no added HCI guidance and a Skill-conditioned generation process.

H1: Interfaces generated with HCI Skills will achieve higher HCI-conformance scores and fewer validated HCI violations than interfaces generated without HCI Skills.

RQ1 tests the central empirical claim. A positive result would support the proposition that procedural HCI knowledge changes generated artifacts in measurable ways. A null or negative result would constrain the claim and identify where the current Skills fail to influence generation.

2.2 RQ2: Value of the Skill Mechanism

RQ2: Does structured Skill-based HCI guidance outperform equivalent HCI guidance supplied through conventional prompting?

A two-condition study cannot separate the value of the Skill mechanism from the value of giving the model more HCI information. The experiment therefore needs a conventional prompting control that receives comparable HCI content without the Skill structure, triggering rules, procedural steps, and verification requirements.

H2: Skill-conditioned generation will achieve higher HCI-conformance scores than conventional HCI prompting when both conditions communicate equivalent design knowledge.

RQ2 is critical to the paper's contribution. If conventional prompting performs as well as the Skills condition, the result would suggest that the HCI content provides value while the current Skill representation adds little. If Skills outperform the prompt control, the study would provide evidence for procedural structure as a useful mechanism.

2.3 RQ3: Reliability and Variance

RQ3: Does Skill-conditioned generation reduce variability in HCI quality across repeated generations?

Generative models are stochastic. A useful software-engineering mechanism should improve the average outcome and make behavior more predictable across repeated runs.

H3: Skill-conditioned generation will exhibit lower within-task, within-model variance in HCI-conformance scores than baseline and conventional prompting conditions.

This question directly tests the proposed uniformity property of Skills [1]. A substantial reduction in variance could be valuable even if the mean-score improvement is modest because production systems require repeatable application of design rules.

2.4 RQ4: Generalizability

RQ4: Are the effects of HCI Skills consistent across different LLMs and UI task types?

H4: HCI Skills will improve conformance across all tested model families, although effect size may vary by model and task type.

RQ4 tests whether the Skill representation behaves like portable design knowledge or depends heavily on one model family. The same analysis can identify task classes that benefit more or less from procedural HCI guidance.

3. Experimental Design

3.1 Factorial Structure

A practical first study can use ten task specifications, three LLM families, three generation conditions, and five independent replications per cell.

Experimental factor	Proposed levels
UI task	10 task specifications
Model	Claude, ChatGPT, Gemini
HCI guidance condition	Baseline, conventional HCI prompt, HCI Skills
Replication	5 independent generations
Total generated interfaces	10 x 3 x 3 x 5 = 450

Five replications create enough repeated observations to measure within-condition variability while keeping the study operationally manageable. **[TODO: run a pilot and power/sensitivity analysis to confirm the final replication count before preregistration.]**

The task specification serves as the repeated experimental unit across models and conditions. Every condition receives the same functional request. The study should vary only the HCI-guidance mechanism and the model under test.

3.2 UI Task Corpus

The task corpus should cover different interaction patterns rather than ten visually similar dashboards. A candidate set includes financial portfolio management, healthcare scheduling, travel planning, product comparison, project management, data analysis, calendar scheduling, smart-home control, course management, and cybersecurity monitoring.

Each task should contain enough interaction complexity to activate several HCI Skills. Tasks should include combinations of navigation, forms, editable objects, destructive or reversible actions, status feedback, filtering, data visualization, and progressive disclosure. Some tasks should include a user profile or accommodation need so that ability-based personalization can be evaluated. Some should include agentic actions so that mixed-initiative principles become applicable.

The corpus should not tell the model how to design the interface. A task should specify goals, domain information, required operations, and data constraints. It should not prescribe layout, control types, accessibility techniques, or HCI principles. This separation prevents the task specification from acting as an uncontrolled design intervention.

[TODO: finalize the ten task specifications and build a task-to-Skill coverage matrix before running the experiment.]

3.3 Model Execution

The study should execute Claude, ChatGPT, and Gemini through a common harness where possible. The harness should standardize the task prompt, framework constraints, available component library, tool access, maximum generation budget, and output format. Exact model versions and inference settings must be recorded because model products change over time.

[TODO: freeze the exact model/API versions, temperature or sampling settings, context limits, tool permissions, and execution date window.]

A common harness also addresses a cross-platform problem: native "Skills" mechanisms may differ across providers. The study should operationalize a Skill as a versioned `skill.md` specification loaded into the controlled agent context through the same orchestration layer for each model. That design tests the Skill artifact as portable procedural knowledge instead of testing three provider-specific product features.

3.4 Generation Conditions

Baseline

The baseline receives the UI task, common implementation constraints, and no additional HCI instruction. The model can use whatever design knowledge is already present in its weights.

Conventional HCI Prompt

The prompt-control condition receives the same task plus HCI guidance expressed as ordinary natural-language instructions. This condition should communicate the same underlying principles contained in the Skill suite while removing the Skill structure, triggers, procedural sequencing, and explicit verification workflow.

The prompt-control condition should be matched as closely as practical on information content. Token count can be reported as a secondary control because a much longer Skill condition could otherwise confound representation with additional context.

[TODO: construct and freeze a content-equivalent conventional HCI prompt; report both semantic coverage and token length relative to the Skill condition.]

HCI Skills

The Skills condition receives the task plus the versioned HCI Skill suite. The initial suite can include Nielsen heuristics, Norman affordances, accessibility/WCAG, ability-based personalization, cognitive-load/progressive-disclosure, mixed initiative, and direct manipulation [1]. The harness should record which Skills were triggered and which verification steps the agent reports applying.

[TODO: freeze the exact Skill files and commit hashes used for the study. Any Skill revision after data collection begins should require a new experimental run.]

3.5 Output Normalization

Every generation should produce the same artifact class, preferably an executable web UI in a fixed framework and component environment. The harness should build and render each interface in an isolated container, reject outputs that do not execute, and capture the source code, DOM, screenshot, interaction trace, and run metadata.

Build failure is itself a measurable outcome and should not be silently discarded. A model that produces higher HCI scores only when its interfaces execute less often should not appear superior because failed outputs were removed from analysis.

4. Agentic Evaluation Framework

The evaluation pipeline should remain identical for all generation conditions. Evaluators must not know which model or HCI-guidance condition produced an interface. File names, comments, prompt traces, and other provenance that reveal condition should be removed from the evaluation package.

The core causal structure is:

> *Task specification -> generation condition -> generated UI -> blinded evaluation -> metrics -> statistical analysis*

The generation intervention changes. The evaluation procedure does not.

4.1 Atomic HCI Criteria

The existing Skills should be decomposed into atomic, testable evaluation criteria. Each criterion should define:

- the HCI principle and source;
- the observable interface evidence required for compliance;
- a scoring rule of satisfied, violated, or not applicable;
- an optional severity level for a validated violation.

For example, a direct-manipulation criterion can ask whether an object that users are expected to manipulate is visible, whether actions are incremental and reversible, and whether direct object manipulation replaces avoidable command syntax [6]. An affordance criterion can ask whether a control's perceived form signals its available action [5]. WCAG criteria can use the corresponding success criteria as the normative source [3].

A criterion should be scored only when it applies to the interface under evaluation. This prevents simple interfaces from receiving inflated scores because they avoid interaction patterns that would create additional obligations.

[TODO: freeze the atomic evaluation rubric before the main run. Validate ambiguous criteria during a pilot rather than modifying scoring rules after observing experimental results.]

4.2 Deterministic Evaluation

Deterministic checks should handle criteria that can be measured directly from code or runtime behavior. Examples include color contrast, missing programmatic labels, invalid ARIA relationships, keyboard reachability, focus behavior, and selected reversible-action requirements. WCAG-oriented tools such as axe-core can support this layer, supplemented by DOM inspection and scripted browser interactions [2,3].

Deterministic checks provide repeatable evidence and reduce dependence on LLM judges. Their coverage will be incomplete because several HCI principles require semantic or contextual interpretation.

4.3 Independent Agent Judges

Semantic criteria should be evaluated by multiple independent agent judges using a fixed rubric. A judge receives the task specification, rendered interface, source code, interaction trace, and atomic evaluation criteria. It does not receive the generation prompt, Skill activation log, model identity, or condition label.

Claude, ChatGPT, and Gemini can each serve as independent judges. Every judge should return structured findings rather than free-form critique. A finding should identify the violated criterion, evidence location, rationale, severity, and confidence.

The system should also require the judge to mark criteria as satisfied or not applicable so that silence is not interpreted as compliance.

Agreement should be measured across judges. Criterion-level agreement can use Krippendorff's alpha, while composite continuous scores can use an intraclass correlation coefficient. A sensitivity analysis should exclude same-family self-judgment, such as removing Claude's evaluation of a Claude-generated interface, to test whether model-family affinity changes the conclusions.

A small author audit can provide an additional quality-control layer without creating a human-subject study. The authors can inspect a preregistered random sample of outputs to estimate false-positive and false-negative rates in the agentic evaluation pipeline.

[TODO: choose the author-audit sample size and adjudication rule before the main run.]

4.4 Primary and Secondary Measures

The primary outcome should be a criterion-based HCI conformance score rather than raw finding count. A simple unweighted score is:

```
HCI Conformance = 100 x satisfied applicable criteria / all applicable criteria
```

This measure is interpretable and avoids making an unvalidated severity weighting the primary endpoint.

Secondary outcomes can include:

- validated violation count per interface;
- severity-weighted violation score;
- conformance by Skill or HCI principle class;
- deterministic-test failure count;
- build/execution success rate;
- within-cell variance across repeated generations.

The study can also report judge disagreement and evaluator false-positive rates. These measures help separate changes in generated UI quality from uncertainty in the measurement system.

5. Analysis Plan

The analysis should treat task as a repeated source of variation rather than pooling all 450 interfaces as independent observations.

5.1 RQ1: Skill Effectiveness

A mixed-effects regression can model HCI-conformance score with guidance condition and model as fixed effects and task as a random effect. The primary planned contrast compares Skills against baseline. The analysis should report effect size and confidence intervals in addition to statistical significance.

Violation counts may require a Poisson or negative-binomial model if the distribution is skewed or overdispersed. The final model family should be selected from the pilot data using preregistered diagnostics.

5.2 RQ2: Skills Versus Prompting

The key planned contrast compares the Skills condition with the conventional HCI prompt condition. This analysis isolates the incremental value of the structured Skill representation after controlling for the presence of HCI guidance.

A strong result for RQ1 combined with a null result for RQ2 would still be useful. It would show that explicit HCI knowledge improves generation while failing to establish that the current Skill mechanism is superior to ordinary prompting. A positive RQ2 result would provide stronger evidence for the procedural representation proposed in the vision paper.

5.3 RQ3: Variance Reduction

For each task-model-condition cell, the study should calculate the spread of conformance scores across replications. A Brown-Forsythe test or a model of absolute deviation from the cell median can compare dispersion across guidance conditions. Reporting within-cell standard deviations and confidence intervals will make the reliability effect easy to interpret.

5.4 RQ4: Cross-Model and Cross-Task Effects

The mixed-effects model should include a model-by-condition interaction. A significant interaction indicates that Skills affect model families differently. Task-level random effects and exploratory condition-by-task analyses can identify domains or interaction patterns where Skills are more effective.

The paper should treat model ranking as secondary. Foundation models change quickly; the durable scientific claim concerns whether a portable HCI knowledge representation affects multiple model families under controlled conditions.

[TODO: preregister the final statistical models, planned contrasts, multiple-comparison correction, exclusion rules, and significance threshold before the main data collection.]

6. Controls That Strengthen Causal Interpretation

The study's value depends on controlling several sources of alternative explanation.

Equivalent tasks. Every generation cell receives the same functional specification. Task wording should be frozen before data collection.

Equivalent implementation environment. Each model should target the same framework, component set, and runtime constraints. This reduces quality differences caused by framework choice.

Information-matched prompt control. The conventional HCI prompt should contain the same substantive design knowledge as the Skills condition. This is the primary control for RQ2.

Blinded evaluation. Evaluators should not know model identity or condition. Blinding reduces both human and agentic expectancy effects.

Versioned artifacts. Prompts, Skills, models, code, rubrics, and evaluation scripts should be versioned. Every generated interface should retain a complete provenance record outside the blinded evaluation package.

Repeated generations. Replications estimate stochastic variation and enable RQ3. One generation should never stand in for a model-condition pair.

Failure retention. Build failures, tool failures, and incomplete outputs should remain in the dataset under preregistered scoring rules.

7. What This Study Can and Cannot Establish

This design can establish whether Skills improve **measured HCI conformance under the selected rubric and automated evaluation environment**. It can test repeatability, cross-model portability, and the incremental value of the Skill representation relative to conventional prompting.

The design cannot establish that users complete tasks faster, make fewer errors, prefer the interface, learn it more quickly, or experience lower cognitive load. Those outcomes require human evidence or a separately validated predictive measure. A result should therefore use language such as "higher HCI-conformance score" rather than "more usable" unless a future user study supports the stronger claim.

This boundary is especially important for cognitive load, affordances, and learnability. Agent judges can score observable design proxies, but they do not turn a proxy into a human behavioral measure. The current study should present those scores as conformance with encoded design criteria.

The evaluation rubric is another source of construct validity risk. If the same research team authors both a Skill and the test used to evaluate it, the evaluation can favor the representation by construction. The study should mitigate this risk by grounding criteria in the original HCI sources, separating generation instructions from evaluation wording, using deterministic checks where possible, and conducting a blinded author audit.

Model drift also limits long-term reproducibility. The study should preserve model identifiers, dates, parameters, and raw outputs. The exact numerical ranking of providers may age quickly; the experimental framework and condition effects should remain reproducible with later model generations.

8. Expected Scientific Contributions

The study can make four contributions even if individual hypotheses are not supported.

First, it provides the first direct empirical test of the central Skills claim from *Automating the Application of HCI Principles* [1]. The result can support, limit, or reject the proposition that HCI knowledge encoded as procedural constraints improves generated interfaces.

Second, it contributes a reusable benchmark for HCI quality in generative UI. The benchmark includes task specifications, versioned Skills, a criterion-level rubric, a blinded agentic evaluation pipeline, and repeated-generation data. Other researchers could use the same infrastructure to compare new models or new HCI representations.

Third, the experiment separates HCI content from HCI representation. RQ2 asks whether a Skill contributes beyond an ordinary prompt that communicates comparable design knowledge. This distinction is necessary if Skills are to be treated as a software-engineering mechanism rather than a convenient packaging format.

Fourth, the study evaluates reliability as a first-class outcome. A generative UI system needs predictable application of HCI constraints across runs. Measuring variance gives the "uniform" property proposed in the vision paper a quantitative test [1].

9. A Broader Research Program

This first study should remain narrow. Its purpose is to determine whether Skills change generated artifacts under controlled conditions. The same infrastructure can support follow-on studies.

A second study could examine which HCI principle classes are most amenable to proceduralization. Accessibility and code-level semantics may be easier to operationalize than aesthetic quality, cognitive load, or domain-language appropriateness.

Measuring effect size by Skill would show where machine-readable HCI knowledge works well and where human judgment remains central.

A later study can connect conformance to human outcomes. A smaller user study could select interfaces from the strongest and weakest experimental conditions and test task completion, error rate, learning, and subjective experience. The agentic study would then serve as a screening experiment that narrows the conditions worth the cost of human evaluation.

The research program follows a repeatable pattern: encode a design proposition, make the proposition measurable, manipulate its presence during generation, collect outputs at scale, and test whether the resulting evidence supports the claim. That pattern converts the original vision paper into a sequence of falsifiable studies.

10. Recommended Execution Sequence

The first implementation can proceed in six phases.

1. Freeze the Skill suite and decompose each Skill into atomic evaluation criteria.
2. Author the ten task specifications and verify coverage across the Skill suite.
3. Build the common generation harness and three guidance conditions.
4. Pilot a small subset to validate execution, evaluator agreement, and scoring behavior.
5. Freeze the protocol, preregister the analysis, and run the complete experiment.
6. Analyze the resulting dataset and report supported, unsupported, and model-dependent effects.

The pilot should be used to fix the instrument rather than to establish the result. Main-study rules should remain fixed after the first full experimental run begins.

11. Conclusion

The HCI Skills concept is ready for a controlled empirical test. A 10-task, three-model, three-condition, repeated-generation design can produce hundreds of interfaces without recruiting a participant pool. A blinded evaluation pipeline can then measure whether Skills improve HCI conformance, outperform ordinary HCI prompting, reduce stochastic variance, and generalize across model families.

The strongest version of this study keeps the claim precise. It does not attempt to prove that Skill-conditioned interfaces are universally "better" for users. It tests whether a versioned procedural representation of HCI knowledge causes measurable improvements in generated artifacts under a predefined evaluation framework. That is a falsifiable claim, and the result remains informative whether the hypotheses are supported or rejected.

References

[1] Nathan Conklin, Miranda Capra, and Chris North. 2026. *Automating the Application of HCI Principles: Skills for On-Demand UI Construction, the Human-AI Space to Think, and the Future of HCI.* FutureHCI '26, Blacksburg, VA.

[2] Maitri Pathak, Yoongi Baek, Hanru Li, Jason Kayne, and Sehrish Basir Nizamani. 2026. *Can LLMs Improve Accessibility? Evaluating LLM Detection and Remediation of WCAG Violations in Web Dashboards.* FutureHCI '26, Blacksburg, VA.

[3] World Wide Web Consortium. 2024. *Web Content Accessibility Guidelines (WCAG) 2.2.* W3C Recommendation.

[4] Jakob Nielsen. 1994. *Heuristic Evaluation.* In *Usability Inspection Methods*. John Wiley & Sons.

[5] Donald A. Norman. 2013. *The Design of Everyday Things*, revised and expanded edition. Basic Books.

[6] Ben Shneiderman. 1983. Direct Manipulation: A Step Beyond Programming Languages. *Computer* 16, 8, 57-69. https://doi.org/10.1109/MC.1983.1654471

[7] Krzysztof Z. Gajos, Daniel S. Weld, and Jacob O. Wobbrock. 2010. Automatically Generating Personalized User Interfaces with Supple. *Artificial Intelligence* 174, 12, 910-950. https://doi.org/10.1016/j.artint.2010.05.005

[8] John Sweller. 1988. Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science* 12, 2, 257-285. https://doi.org/10.1207/s15516709cog1202_4

[9] Eric Horvitz. 1999. Principles of Mixed-Initiative User Interfaces. In *Proceedings of CHI '99*, 159-166. https://doi.org/10.1145/302979.303030